SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	**Assessment:**	Industry Problem solving task
Weightage:	30%	**Total Marks:**	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	**ABDUL WAHID. H**	**Reg. No.:**	**192321079**
Section / Batch:	**B.TECH IT**	**Date:**	**19/08/2026**

Instructions to Students

- Identify the relevant concepts of cache hierarchy, SRAM, DRAM, MRAM, NAND Flash, RRAM, memory latency, bandwidth, and virtual memory paging.
- Analyze the memory-access requirements of smartphones, cloud servers, autonomous vehicles, edge AI devices, and gaming consoles, considering latency, bandwidth, capacity, and power consumption.
- Apply suitable memory-hierarchy techniques such as cache optimization, appropriate memory technology selection, data locality, paging optimization, and hierarchical memory organization.
- Compute performance measures such as memory access time, latency, bandwidth, throughput, speedup, hit/miss rates, and resource or power utilization where applicable.
- Interpret the results by evaluating performance improvements, memory bottlenecks, technology trade-offs, and the suitability of the proposed memory hierarchy for each application.

QUESTIONS

1. A mobile device manufacturer observes noticeable lag when users switch rapidly between multiple applications. Analyze how L1, L2, and L3 caches and DRAM participate in storing and retrieving frequently accessed application data. Based on latency, capacity, hit rate, and bandwidth requirements, propose a suitable hierarchical memory organization that can reduce applicationswitching delays and improve overall system throughput.
2. A cloud service provider experiences increased latency when processing database queries from a large number of concurrent users. Analyze how cache hierarchy and virtual memory paging influence memory access performance, and compare their roles in reducing processor-memory latency. Recommend suitable memory technologies and memory-management improvements that can minimize page faults, reduce access delays, and improve database throughput.
3. An autonomous vehicle continuously receives large volumes of sensor data from cameras, LiDAR, radar, and other real-time sensing systems. Evaluate the characteristics of SRAM, DRAM, and MRAM in terms of access latency, bandwidth, capacity, volatility, and power consumption. Based on the real-time processing requirements, design an appropriate memory hierarchy that can provide high data-transfer bandwidth while minimizing access latency and supporting reliable sensor-data processing.
4. An edge-based AI device must load machine-learning models quickly while operating under strict power and storage constraints. Analyze the characteristics of NAND Flash, DRAM, and RRAM with respect to access speed, storage capacity, persistence, power consumption, and suitability for AI workloads. Recommend a hierarchical memory organization that enables fast model loading, efficient intermediate-data processing, reduced energy consumption, and improved overall inference performance.
5. A high-performance gaming console experiences frame drops and noticeable delays when loading large game environments containing high-resolution textures, audio, and other assets. Analyze the interaction between L3 cache and DRAM during the retrieval and processing of frequently accessed game data, considering their latency, capacity, and bandwidth. Propose suitable

memoryhierarchy enhancements that can reduce data-access bottlenecks, increase memory throughput, and provide smoother frame rendering during large scene transitions.

Code of Conduct

*I **ABDUL WAHID. H / 192321079** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ABDUL WAHID. H

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Q. No	Understanding of Industry Problem (20)	Application of Engineering Concepts (25)	Solution Design (25)	Use of Tools (15)	Analysis (10)	Professionalism(5)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/5	/ 100

Course Coordinator **Faculty Incharge**

1. Mobile device – reducing application-switching delays

Role of each level

- L1 cache is the smallest and fastest cache and is located closest to the CPU core. It stores the most frequently and recently accessed instructions and data. A high L1 hit rate allows the CPU to continue execution with very little waiting.
- L2 cache is larger than L1 but slightly slower. It acts as the next level when an L1 miss occurs and reduces the number of requests that must reach slower shared memory.
- L3 cache, when implemented, is usually larger and shared across multiple CPU cores. It can hold commonly shared application data and reduce expensive DRAM accesses.
- DRAM provides the large main-memory capacity needed to keep multiple applications active. It has much higher capacity than cache but substantially higher access latency.

Recommended hierarchy

- Use separate instruction and data L1 caches for each CPU core, followed by a private or tightly coupled L2 cache, and a larger shared L3 cache where the processor supports it.
- Use high-bandwidth, low-power mobile DRAM as the main memory. Keep active application working sets in DRAM so that application switching does not repeatedly require storage access.
- Use cache-aware software techniques such as data locality, compact working sets, prefetching where beneficial, and avoidance of unnecessary memory copies.
- The operating system should keep recently used application pages resident in DRAM and avoid aggressive swapping. Memory compression can also help maintain more active application data in RAM when physical memory is constrained.

Why this reduces lag

- Most accesses should be satisfied by L1/L2/L3 because cache hits have much lower latency than DRAM accesses. A high cache hit rate therefore reduces CPU stalls.
- L3 acts as an important buffer for data shared among cores and reduces pressure on DRAM bandwidth.
- Keeping active application state in DRAM avoids repeated loading from slower persistent storage during application switching.
- Overall throughput improves because CPU cycles are spent executing instructions rather than waiting for data.

Conclusion

- A practical mobile hierarchy is: CPU registers → L1 → L2 → shared L3 → high-bandwidth DRAM → persistent storage. The design should maximize locality and cache hit rate while providing sufficient DRAM capacity and bandwidth for multiple applications.

2. Cloud server – database query latency

Cache hierarchy

- CPU caches reduce the effective latency of frequently accessed database instructions and data. L1 provides the fastest access, L2 provides a larger backup cache, and L3 provides a still larger shared cache.
- Database workloads benefit from temporal locality, such as repeatedly accessing indexes or frequently used records, and spatial locality, such as scanning adjacent records.
- A high cache hit rate reduces the number of accesses that must proceed to DRAM, thereby reducing CPU stalls and memory-system traffic.

Virtual memory and paging

- Virtual memory allows each process to use a virtual address space larger or more flexible than the immediately available physical-memory arrangement. Pages are mapped to physical frames.
- If a required page is not in physical memory, a page fault occurs. The operating system must retrieve the page from secondary storage, which is far slower than DRAM. Frequent page faults can therefore dominate database response time.
- Paging is fundamentally different from cache hierarchy: caches transparently reduce CPU-to-memory access latency for frequently used blocks, while virtual memory manages the mapping and movement of larger pages between physical memory and storage.

Recommended technologies and improvements

- Use high-capacity, high-bandwidth DRAM as the main working memory for database buffer pools, indexes, query execution structures, and frequently accessed records.
- Use large shared CPU caches where available and tune database data structures to improve locality.
- Allocate sufficient RAM to keep the database working set and buffer pool resident. Avoid memory overcommit that causes excessive swapping.
- Use huge pages where appropriate to reduce translation overhead and TLB misses, provided the database and operating system configuration support them.
- Use efficient indexing, query optimization, connection pooling, and partitioning so that fewer data blocks must be touched for each query.
- Monitor page-fault rate, cache hit rate, memory utilization, I/O wait, and query latency. The target should be a very low major-page-fault rate for latency-sensitive database workloads.

Conclusion

- Cache hierarchy primarily reduces processor-memory latency, while virtual memory provides protection, isolation, and flexible memory management. For cloud databases, the best strategy is to maximize cache locality, keep the active database working set in DRAM, minimize page faults, and prevent unnecessary paging to storage.

3. Autonomous vehicle – real-time sensor processing

SRAM

- SRAM offers very low access latency and high performance, making it suitable for processor caches, small buffers, queues, and real-time control data.
- It is volatile, consumes more silicon area per bit than DRAM, and is therefore much less suitable for very large-capacity storage.
- Its high speed is valuable when a sensor-processing loop has strict timing constraints.

DRAM

- DRAM provides much greater capacity than SRAM and is suitable for large camera frames, LiDAR point clouds, radar data, intermediate AI tensors, and operating-system/application working sets.
- It is volatile and generally slower than SRAM, but modern DRAM provides high bandwidth when used with wide memory interfaces and parallel channels.
- Power must be managed carefully because continuous high-bandwidth sensor processing can create substantial memory traffic.

MRAM

- MRAM is non-volatile and retains information when power is removed. It offers fast access compared with many conventional non-volatile memories and can provide useful endurance.
- MRAM can be useful for persistent configuration, critical state, logs, and selected fast-access buffers where non-volatility is valuable.
- For very large sensor streams, DRAM generally remains the practical high-capacity working memory, while MRAM can complement it for selected persistent or reliability-sensitive data.

Recommended hierarchy

- Use CPU/GPU/AI-accelerator registers and SRAM-based caches at the top for immediate processing.
- Use dedicated SRAM scratchpads and double buffers close to sensor-processing accelerators to absorb bursts and support deterministic real-time processing.
- Use high-bandwidth DRAM as the main working memory for camera, LiDAR, radar, and AI intermediate data.
- Use MRAM for selected persistent system state, calibration/configuration information, event logs, or other data that benefits from non-volatility.
- Apply DMA, double buffering, memory pooling, data compression where appropriate, and zero-copy data paths to reduce CPU overhead and unnecessary memory transfers.

Conclusion

- The most suitable hierarchy is SRAM for ultra-low-latency processing, high-bandwidth DRAM for large real-time sensor datasets, and MRAM for selected persistent and reliability-sensitive information. This combination balances latency, bandwidth, capacity, volatility, and power.

4. Edge AI device – fast model loading under power constraints

NAND Flash

- NAND Flash is non-volatile, so model files remain stored when the device is powered off. It offers high storage density and is suitable for storing large AI models and application software.
- Its access latency is much higher than DRAM, so it is not ideal for repeatedly accessing individual model weights during inference.
- It is therefore best treated as persistent model storage rather than the main inference working memory.

DRAM

- DRAM is volatile but provides substantially faster random access and much higher suitability for active AI inference workloads than NAND Flash.
- Model weights, input tensors, activation maps, and intermediate results can be placed in DRAM during inference.
- Its capacity is lower than high-density NAND Flash, and keeping large amounts of DRAM active consumes power, so memory placement and data movement should be optimized.

RRAM

- RRAM is a non-volatile resistive memory technology with potential for high density, low standby power, and fast access compared with conventional storage technologies.
- Its emerging characteristics make it attractive for edge AI, particularly for near-memory or in-memory computing approaches in which weights can be kept close to computation.
- Because practical RRAM implementations vary, its exact endurance, write behavior, density, and performance depend on the device technology.

Recommended hierarchy

- Store the complete model and application data in NAND Flash because it provides persistent, high-density storage.
- At startup, copy the model or the most frequently used model portions into DRAM for fast inference access.
- Where supported by the hardware, use RRAM for frequently reused model weights or near-memory/in-memory AI operations to reduce data movement and standby power.
- Use on-chip SRAM/cache in the AI accelerator for the hottest weights, activations, and intermediate tensors.
- Use model quantization, pruning, compression, batching where appropriate, and tiled execution to reduce memory footprint and energy consumption.

Conclusion

- A suitable hierarchy is: accelerator SRAM/cache → DRAM → RRAM for selected persistent/high-reuse AI data → NAND Flash for large persistent storage. This minimizes slow storage accesses, reduces unnecessary data movement, and supports faster, more energy-efficient inference.

5. Gaming console – reducing frame drops during large scene transitions

L3 cache and DRAM interaction

- L3 cache stores frequently reused data and instructions so that the CPU can access them with lower latency than DRAM. It is smaller than DRAM but much faster.
- When game data is found in L3, the processor avoids a DRAM access. An L3 miss causes the request to proceed to the lower memory hierarchy and potentially DRAM.
- Large game environments can exceed cache capacity, so textures, geometry, audio buffers, and other assets must be managed carefully in DRAM. High DRAM bandwidth is important because modern games continuously stream large amounts of data.

Main bottlenecks

- Frame drops can occur when the CPU/GPU waits for required data, when DRAM bandwidth becomes saturated, or when asset streaming and decompression compete with rendering for memory bandwidth.
- Poor locality causes useful data to be evicted from cache too quickly, increasing cache misses and DRAM traffic.
- Loading too much unused data into memory wastes bandwidth and increases pressure on caches and DRAM.

Recommended enhancements

- Use a larger and well-designed shared L3 cache where the processor architecture permits it, especially for frequently reused CPU-side game data and engine structures.
- Use high-bandwidth DRAM with sufficient capacity to hold active game assets and reduce repeated transfers from slower persistent storage.
- Implement asynchronous asset streaming so that upcoming textures, geometry, and audio are loaded before the player reaches the corresponding scene.
- Use DMA and dedicated decompression hardware where available so that asset movement and decompression do not unnecessarily occupy CPU execution resources.
- Apply prefetching carefully for predictable access patterns and organize game data to improve spatial and temporal locality.
- Use double buffering or ring buffers for streaming assets so that one buffer can be consumed while another is filled in the background.
- Prioritize visible and near-future assets, reducing unnecessary memory transfers during scene transitions.

Conclusion

- The key is to keep the hottest game-engine data in L3 while using high-bandwidth DRAM as the main working store for active assets. Predictive streaming, DMA, buffering, good locality, and efficient decompression can reduce memory stalls and make frame delivery more consistent.